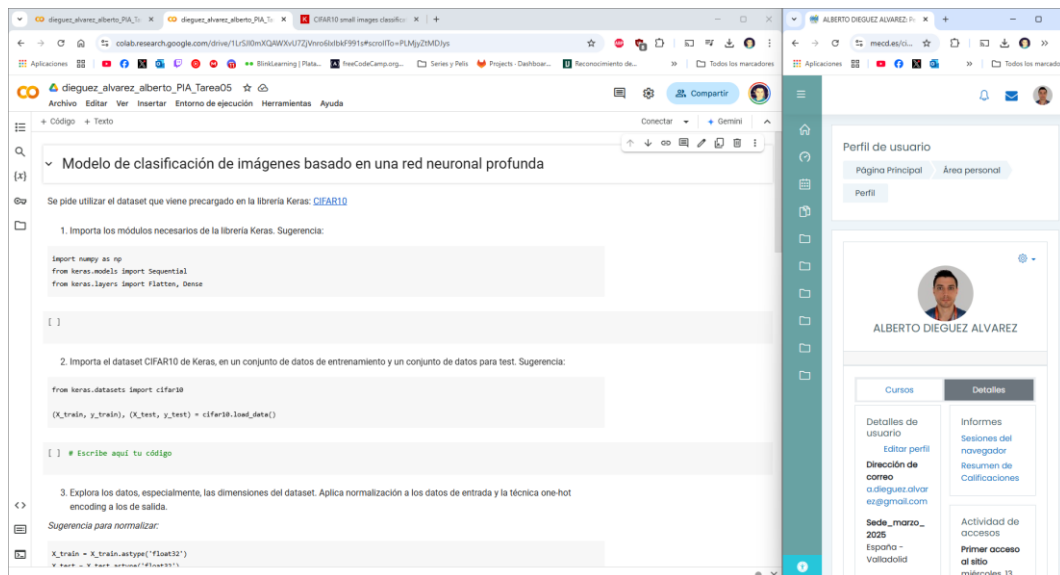


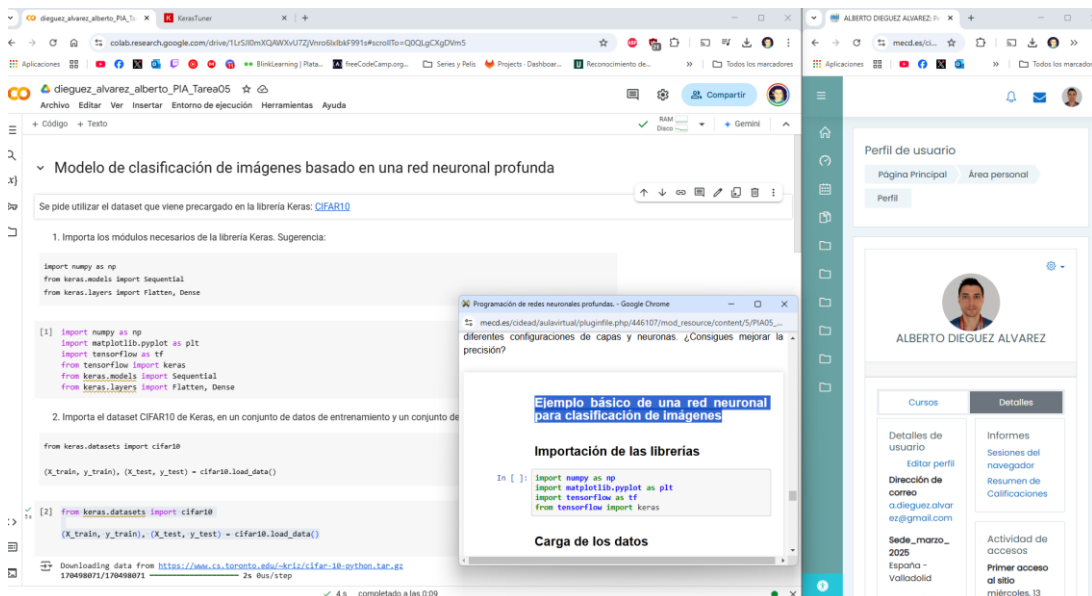
Apartado 1: Carga y explora el dataset CIFAR10

- Inicia un nuevo notebook, preferiblemente en Google Colab. Para guiarte en el proceso, puedes utilizar este [cuaderno-guía](#) con algunas sugerencias de fragmentos de código indicados en las celdas de texto, pero tendrás que escribir el código en la celda de código correspondiente y ejecutarlo.
- Importa la librería **Numpy**.
- Importa los módulos necesarios para construir una red neuronal profunda: Sequential, Dense y Flatten.

Entro en el enlace de la guía, creo un nuevo Google Colab y pego el contenido de la guía, también le cambio el nombre.



Basandome en el apartado 4 - Entrenamiento de la red profunda añado todas las librerías.



Apartado 2: Importa el dataset CIFAR10 de Keras, en un conjunto de datos de entrenamiento y un conjunto de datos para test.

- Consulta la [documentación de Keras relativa a este dataset](#) para entender cómo están organizados los datos y saber importarlos.

The screenshot shows a Google Colab notebook on the left and a web browser on the right. The notebook contains code for importing the CIFAR10 dataset from Keras and normalizing it. The browser shows the Keras documentation for the CIFAR10 dataset, including the `load_data` function and a table of classes.

```
from keras.datasets import cifar10
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

[5]: from keras.datasets import cifar10
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

3. Explora los datos, especialmente, las dimensiones de los datos de entrada y salida.
encoding a los de salida.

Superficie para normalizar:

X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
X_train = X_train / 255.0
X_test = X_test / 255.0

Superficie de aplicación one-hot encoding para categorías:

# one hot encode outputs
y_train = np_utils.to_categorical(y_train)
y_test = np_utils.to_categorical(y_test)
num_classes = y_test.shape[1]

[ ] # Escribe aquí tu código
```

The browser shows the Keras documentation for the CIFAR10 dataset. The page title is "CIFAR10 small images classification dataset". The `load_data` function is highlighted. The documentation states: "Loads the CIFAR10 dataset. This is a dataset of 50,000 32x32 color training images and 10,000 test images, labeled over 10 categories. See more info at the [CIFAR homepage](#)." The classes are listed in a table:

Label	Description
0	airplane
1	automobile
2	bird
3	cat

Apartado 3: Explora los datos.

- Explora los datos, especialmente, las dimensiones del dataset.

Exploro los datos.

The screenshot shows a Google Colab notebook with the following code and output:

```
# one-hot encode outputs
y_train = np_utils.to_categorical(y_train)
y_test = np_utils.to_categorical(y_test)
num_classes = y_test.shape[1]
```

Output:

```
[8] X_train.shape
(50000, 32, 32, 3)

[9] X_test.shape
(10000, 32, 32, 3)

[10] y_train.shape
(50000, 1)

[11] y_test.shape
(10000, 1)
```

A Chrome window titled "Programación de redes neuronales profundas - Google Chrome" is open, showing the Fashion-MNIST dataset details:

```
In [ ]: train_images.shape
Out[ ]: (60000, 28, 28)

In [ ]: len(train_labels)
Out[ ]: 60000

In [ ]: train_labels
Out[ ]: array([9, 0, 0, ..., 3, 0, 5], dtype=uint8)

In [ ]: class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

In [ ]: plt.figure()
plt.imshow(train_images[10])
```

- Aplica normalización a los datos de entrada.

The screenshot shows a Google Colab notebook with the following code and output:

```
[15] X_test.shape # (10000, 32, 32, 3) = 10,000 imágenes de 32x32 píxeles con 3 canales (RGB)
(10000, 32, 32, 3)

[16] # Verificar la forma de las etiquetas de entrenamiento
y_train.shape # (50000, 1) = 50,000 etiquetas de clase, una por imagen
(50000, 1)

[17] # Verificar la forma de las etiquetas de prueba
y_test.shape # (10000, 1) = 10,000 etiquetas de clase, una por imagen
(10000, 1)
```

A Chrome window titled "Programación de redes neuronales profundas - Google Chrome" is open, showing the Fashion-MNIST dataset details:

```
In [ ]: train_images.shape
Out[ ]: (60000, 28, 28)

In [ ]: len(train_labels)
Out[ ]: 60000

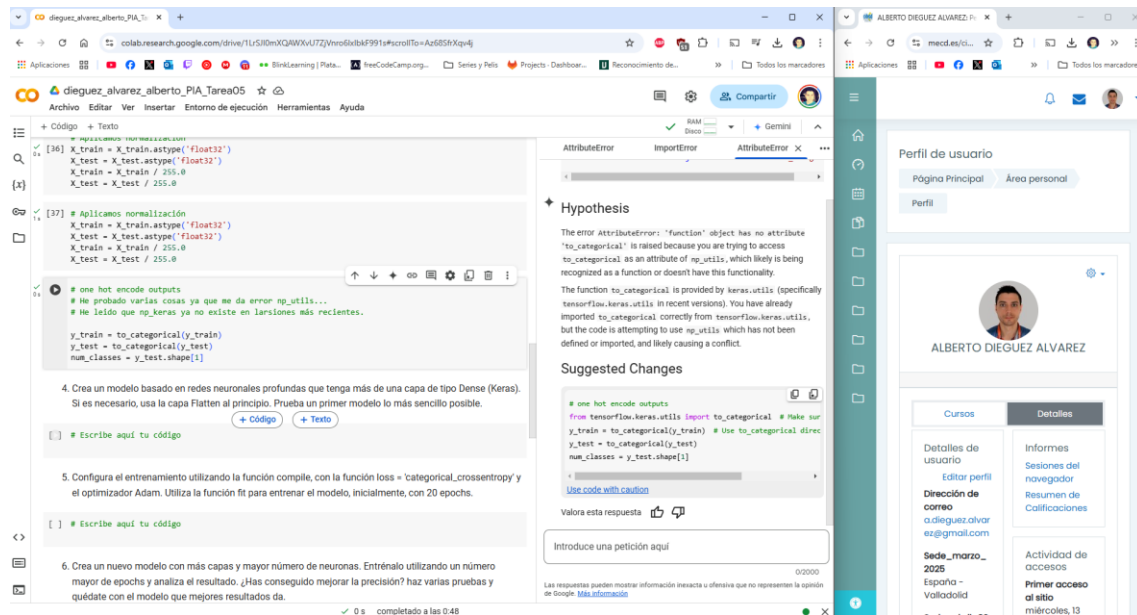
In [ ]: train_labels
Out[ ]: array([9, 0, 0, ..., 3, 0, 5], dtype=uint8)

In [ ]: class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

In [ ]: plt.figure()
plt.imshow(train_images[10])
```

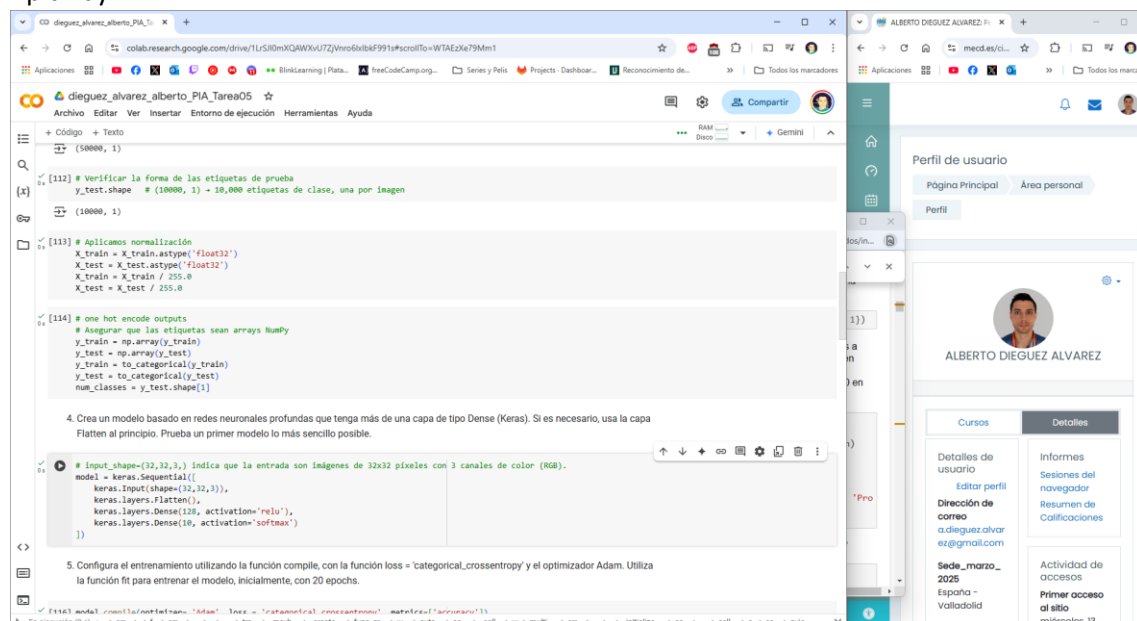
- Aplica la técnica one-hot encoding al conjunto de datos de salida.

Para esta parte, importo `to\_canonical` from keras, y elimino la parte de np_utils ya que me daba error y leí que ya no se usa.



- En general, aplica las funciones necesarias para entender cómo son los datos para poder crear el modelo de forma adecuada y entender también los resultados del entrenamiento.

Al acabar la tarea no se me veían las fotos... y rehaciendo el código lo tuve que hacer con np.array.



Apartado 4: Crea el modelo.

- Genera un modelo con la clase Sequential.

The screenshot displays a Jupyter Notebook environment with a code cell and its output. The code defines a Keras model with three layers: an input layer of size (32, 32, 3), a flattened layer, and a dense output layer of size 10. The output shows the model's architecture and a warning message about the 'input_shape' argument.

```
[37]: # aplicamos normalización
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
X_train = X_train / 255.0
X_test = X_test / 255.0

[42]: # one hot encode outputs
# He probado varias cosas ya que me da error np_utils....
# He leído que np_utils ya no existe en las versiones más recientes.

y_train = to_categorical(y_train)
y_test = to_categorical(y_test)
num_classes = y_test.shape[1]
```

4. Crea un modelo basado en redes neuronales profundas que tenga más de una capa de tipo Dense Flatten al principio. Prueba un primer modelo lo más sencillo posible.

```
model = keras.Sequential([
    # input_shape=(32,32,3), indica que la entrada son imágenes de 32x32 píxeles
    # con 3 canales de color (RGB).
    keras.layers.Flatten(input_shape = (32,32,3)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
```

Warning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When us
super().__init__(**kwargs)

5. Configura el entrenamiento utilizando la función compile, con la función loss = 'categorical_crossentropy' y el optimizador Adam. Utiliza la función fit para entrenar el modelo, inicialmente, con 20 epochs.

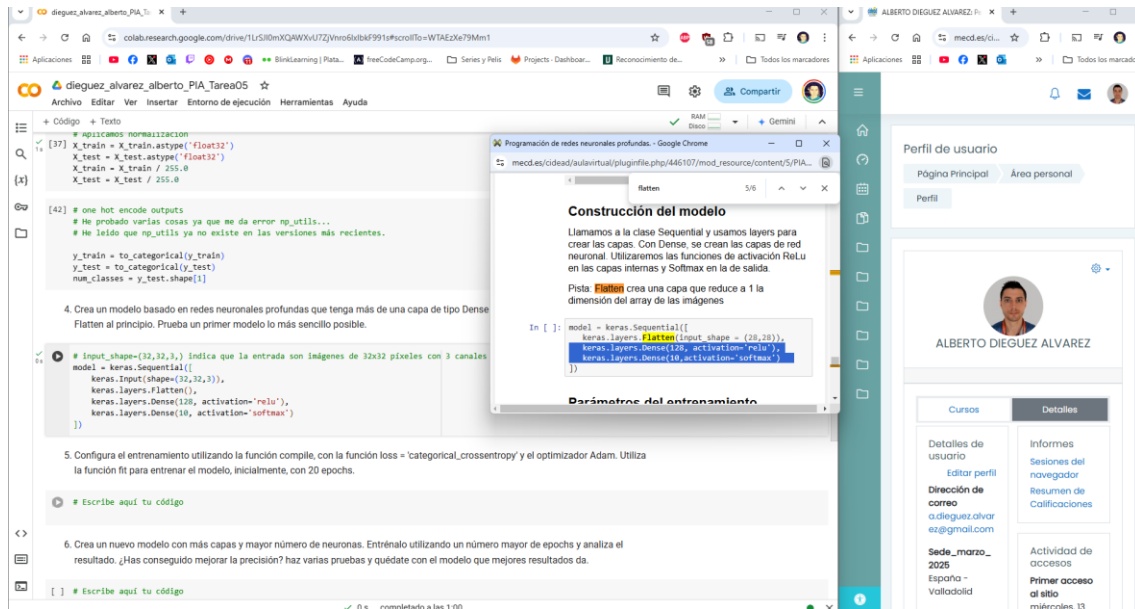
```
[ ] # Escribe aquí tu código
```

6. Crea un nuevo modelo con más capas y mayor número de neuronas. Entrenalo utilizando un número mayor de épocas y analiza el

0 s completado a las 0:57

The web browser on the right shows a user profile for ALBERTO DIEGUEZ ALVAREZ with navigation links for 'Página Principal', 'Área personal', 'Perfil', 'Cursos', and 'Detalles'.

Como la salida me da error, investigo para quitar el warning. Busco información y recomiendan separar el input de dentro del Flatten.

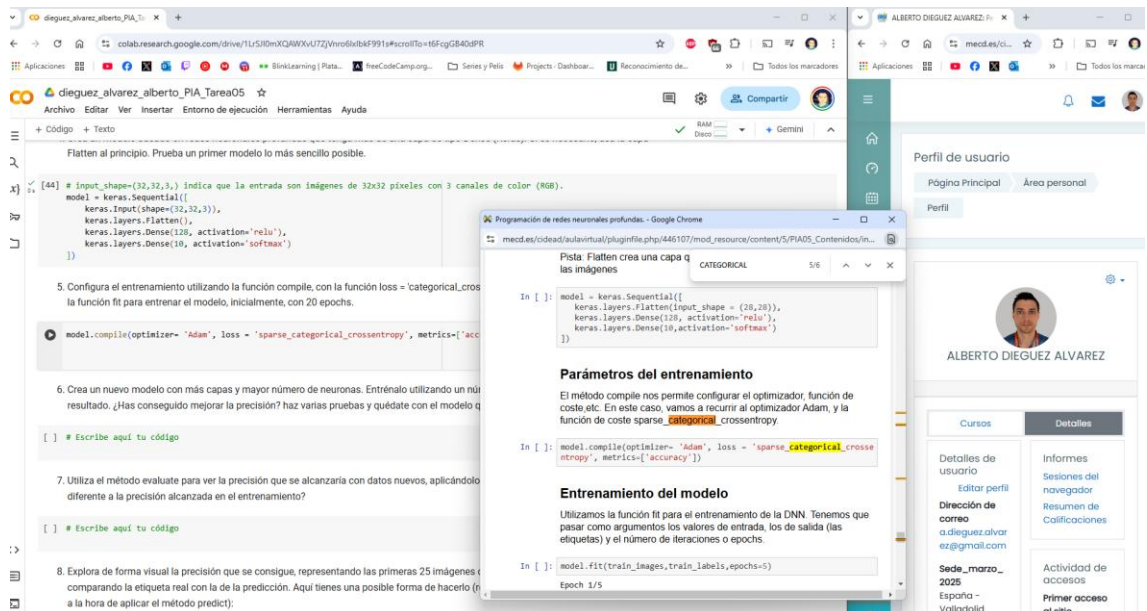


- Añade el menor número de capas posible, utilizando las clases Dense y Flatten.

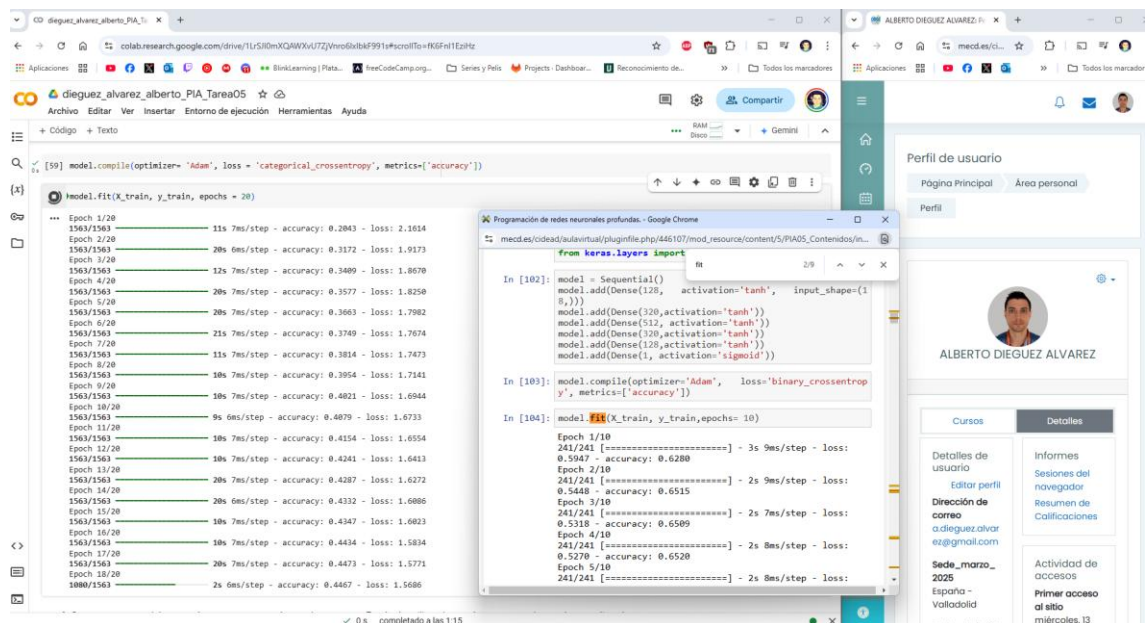
Apartado 5: Entrena el modelo.

- Configura el modo de entrenamiento con el método compile.
- Utiliza la función loss = 'categorical_crossentropy'.
- Selecciona el optimizador Adam.
- Utiliza la función fit para entrenar el modelo, con un máximo de 20 epochs.

Añado el optimizador.



Ejecutamos el modelo.



Apartado 6: Mejora el modelo.

- Crea un nuevo modelo con más capas y mayor número de neuronas.
- Entrénalo utilizando un número mayor de epochs y analiza el resultado.
- ¿Has conseguido mejorar la precisión? haz varias pruebas y quédate con el modelo que mejores resultados da.

Creo un modelo con más capas y neuronas. He visto que se puede modificar el rate del optimizador, pero no lo he cambiado.

dieguez_alvarez_alberto_PIA_Tarea05

6. Crea un nuevo modelo con más capas y mayor número de neuronas. Entrénalo utilizando un número mayor de epochs y analiza el resultado. ¿Has conseguido mejorar la precisión? haz varias pruebas y quédate con el modelo que mejores resultados da.

```
model = keras.Sequential([
    keras.Input(shape=(32,32,3)),
    keras.layers.Flatten(),
    keras.layers.Dense(512, activation='relu'), # Capa densa con 512 neuronas
    keras.layers.Dense(256, activation='relu'), # Capa densa con 256 neuronas
    keras.layers.Dense(128, activation='relu'), # Capa densa con 128 neuronas
    keras.layers.Dense(10, activation='softmax') # Capa de salida
])

model.compile(optimizer='Adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs = 50)
```

Epoch 1/50
354/1563 — 27s 23ms/step - accuracy: 0.1510 - loss: 2.2878

7. Utiliza el método evaluate para ver la precisión que se alcanzaría con datos nuevos, aplicándolo al conjunto de datos de test. ¿Es muy diferente a la precisión alcanzada en el entrenamiento?

Tarda bastante... por lo que veo si que mejora la precisión y baja la pérdida.

dieguez_alvarez_alberto_PIA_Tarea05

Para recibir alertas cuando la ejecución finalice, activa las notificaciones del navegador en Configuración.

model.compile(optimizer='Adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs = 50)

Epoch 1/50
1563/1563 — 36s 22ms/step - accuracy: 0.2189 - loss: 2.0614
Epoch 2/50
1563/1563 — 33s 21ms/step - accuracy: 0.3568 - loss: 1.7794
Epoch 3/50
1563/1563 — 34s 22ms/step - accuracy: 0.3898 - loss: 1.6859
Epoch 4/50
1563/1563 — 41s 22ms/step - accuracy: 0.4144 - loss: 1.6177
Epoch 5/50
1563/1563 — 34s 22ms/step - accuracy: 0.4484 - loss: 1.5551
Epoch 6/50
1563/1563 — 41s 22ms/step - accuracy: 0.4513 - loss: 1.5217
Epoch 7/50
1563/1563 — 40s 22ms/step - accuracy: 0.4700 - loss: 1.4703
Epoch 8/50
1563/1563 — 42s 22ms/step - accuracy: 0.4821 - loss: 1.4354
Epoch 9/50
1563/1563 — 34s 22ms/step - accuracy: 0.4996 - loss: 1.4003
Epoch 10/50
1563/1563 — 42s 22ms/step - accuracy: 0.5042 - loss: 1.3835
Epoch 11/50
1563/1563 — 41s 22ms/step - accuracy: 0.5137 - loss: 1.3502
Epoch 12/50
1563/1563 — 41s 22ms/step - accuracy: 0.5260 - loss: 1.3228
Epoch 13/50
1563/1563 — 35s 22ms/step - accuracy: 0.5329 - loss: 1.3038
Epoch 14/50
1563/1563 — 41s 23ms/step - accuracy: 0.5384 - loss: 1.2879
Epoch 15/50
1243/1563 — 7s 23ms/step - accuracy: 0.5491 - loss: 1.2618

7. Utiliza el método evaluate para ver la precisión que se alcanzaría con datos nuevos, aplicándolo al conjunto de datos de test. ¿Es muy diferente a la precisión alcanzada en el entrenamiento?

Apartado 7: Evalúa el nuevo modelo.

- Utiliza el método evaluate para ver la precisión que se alcanzaría con datos nuevos, aplicándolo al conjunto de datos de test.
- ¿Es muy diferente a la precisión alcanzada en el entrenamiento?

The screenshot displays a Google Colab notebook titled 'dieguez_alvarez_alberto_PIA_Tarea05'. The notebook is open to a cell showing the training progress of a model. The progress bar indicates that the training is complete, with a final accuracy of 0.7333 and a loss of 0.7477. The notebook also includes a section for evaluating the model on a test set, where the accuracy is 0.5368 and the loss is 1.5171. The notebook is running on a GPU (Tesla V100) and has a total runtime of 1 hour and 54 minutes.

7. Utiliza el método evaluate para ver la precisión que se alcanzaría con datos nuevos, aplicándolo al conjunto de datos de test. ¿Es muy diferente a la precisión alcanzada en el entrenamiento?

```
[68] loss, accuracy = model.evaluate(X_test, y_test)
print("Pérdida en el conjunto de datos de prueba:", loss)
print("Precisión en el conjunto de datos de prueba:", accuracy)
```

313/313 2s 5ms/step - accuracy: 0.5324 - loss: 1.5171
Pérdida en el conjunto de datos de prueba: 1.5279818773269653
Precisión en el conjunto de datos de prueba: 0.5368000133514484

8. Explora de forma visual la precisión que se consigue, representando las primeras 25 imágenes del conjunto de datos de test, y comparando la etiqueta real con la de la predicción. Aquí tienes una posible forma de hacerlo (recuerda poner el nombre de tu modelo a la hora de aplicar el método predict):

```
predictions = model.predict(X_test)

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

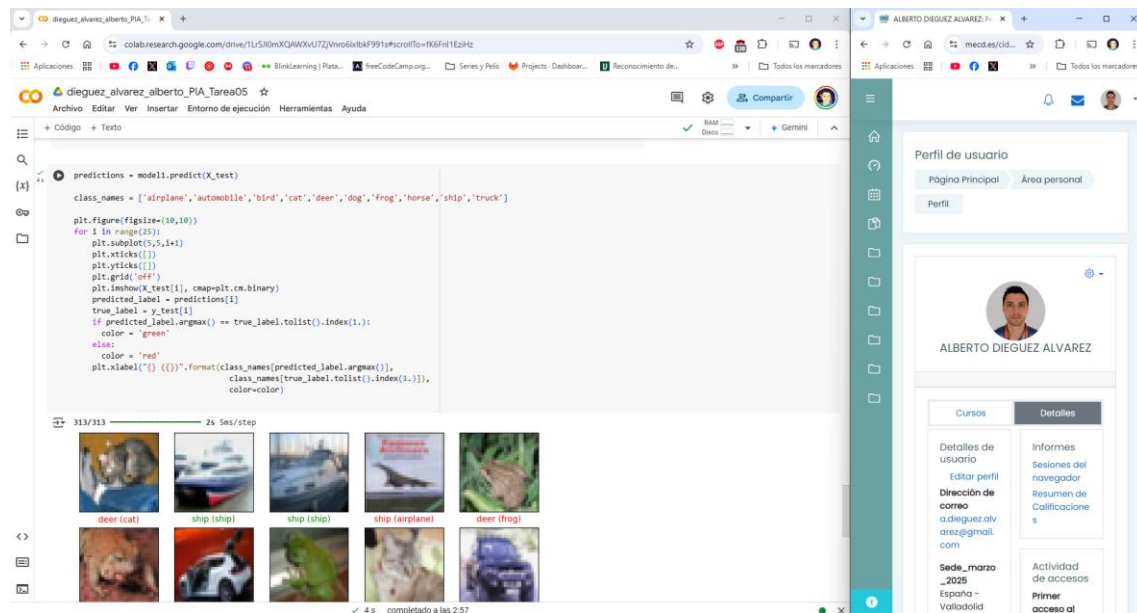
plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.title('%s\n%s' % (class_names[predictions[i]], class_names[y_test[i]]))
```

Sí, mucha diferencia a la precisión del entrenamiento.

Apartado 8: Visualiza las predicciones.

- Explora de forma visual la precisión que se consigue, representando las primeras 25 imágenes del conjunto de datos de test, y comparando la etiqueta real con la de la predicción.
- En la guía tienes un script sugerido para ayudarte con el código.

Ejecuto el script.



No entiendo del todo el script, pero me parece muy interesante.

URL del cuaderno: [Google Colab Notebook](#)